

«ТОП УРОВЕНЬ»

ЧЕМПИОНАТ РОСТЕЛЕКОМА
ДЛЯ СТУДЕНТОВ ТОП-ИТ И ТОП-ИИ





Прототип RAG-системы: векторный поиск по коду с веб-интерфейсом

CodeLens — умный поиск по кодовой базе

• Уровень сложности	Средний
• Направление	Искусственный интеллект / Data Engineering
• Состав команды	от 2 до 4 участников
• Продолжительность	2-й этап чемпионата (20.05 – 24.06.2026)
• Формат защиты	Демонстрация работающего прототипа + защита в ВКС (10 мин.)





Прототип RAG-системы: векторный поиск по коду с веб-интерфейсом

CodeLens — умный поиск по кодовой базе

Контекст задачи

Крупная IT-компания управляет десятками репозиторий. Новичок тратит 2-6 недель, чтобы разобраться где что реализовано. Опытные сотрудники ежедневно отвечают на повторяющиеся вопросы вместо того, чтобы решать новые задачи.

Стандартные инструменты поиска по коду (grep, полнотекстовый поиск) задачу решают плохо. Запрос «как здесь обрабатываются ошибки авторизации» не даст результата, потому что ни одно слово не совпадёт с именами функций в коде.

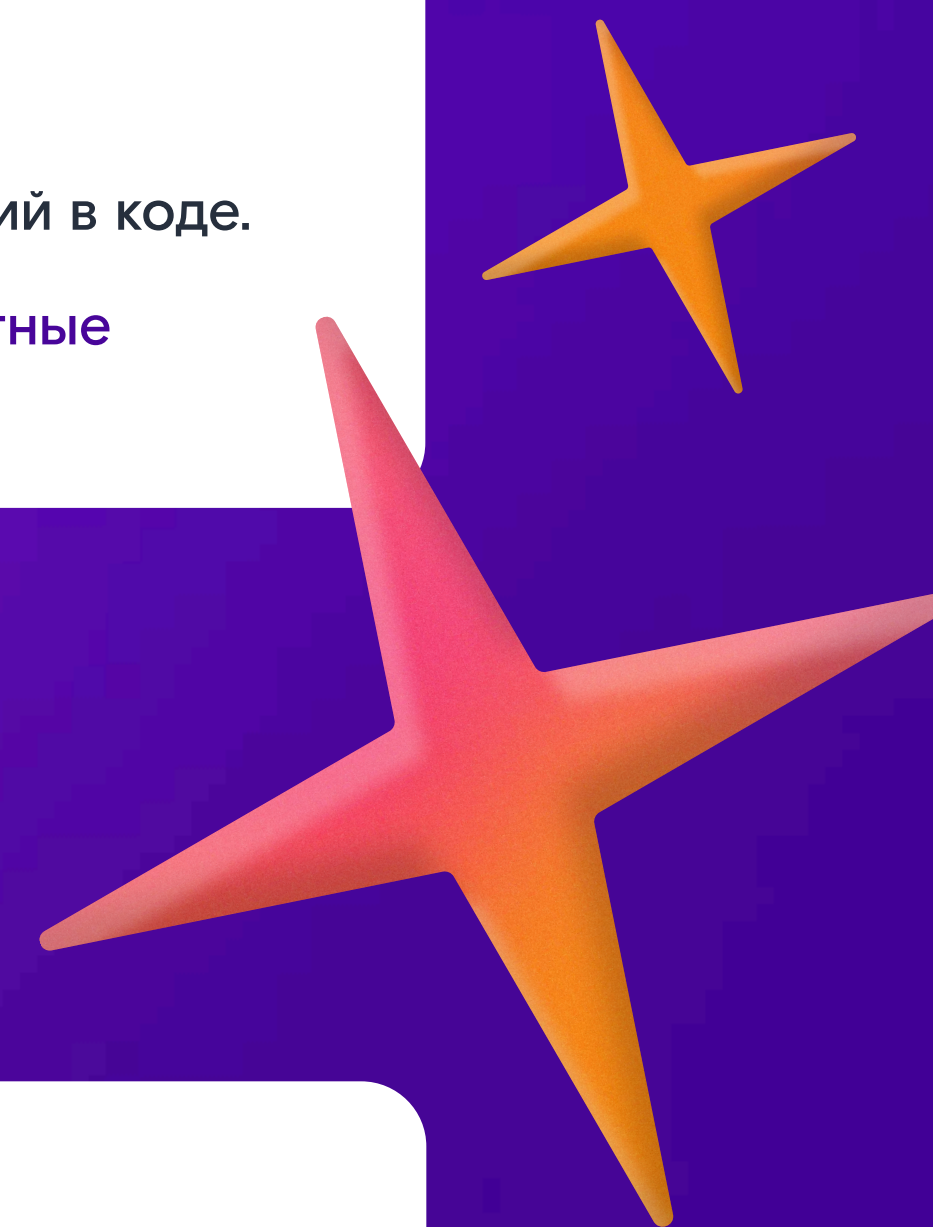
Задача команды — прототип системы, которая понимает вопросы о коде на естественном языке и находит релевантные фрагменты, даже если слова вопроса не совпадают с именами переменных и функций.

Подсказка: RAG (Retrieval-Augmented Generation) — это просто «найди похожее + (опционально) сгенерируй ответ». Минимальный RAG обходится без LLM: только векторный поиск и красивый UI. LLM-часть — это бонусные баллы.

Бизнес-задача

Разработать прототип, который:

- Индексирует Python-кодую базу: режет файлы на функции/классы, строит для каждого эмбединг, сохраняет в векторную БД.
- Выполняет семантический поиск: по вопросу на естественном языке возвращает топ-K релевантных фрагментов кода.
- Предоставляет веб-интерфейс на Streamlit: поле ввода, список результатов с подсветкой синтаксиса, оценкой релевантности.
- Опционально формирует связный ответ: найденные фрагменты + вопрос → LLM → человекочитаемое объяснение.





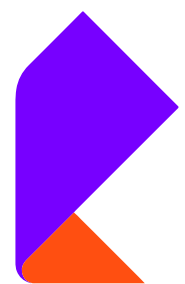
Прототип RAG-системы: векторный поиск по коду с веб-интерфейсом

CodeLens — умный поиск по кодовой базе

Измеримые критерии успеха

Метрика	Целевое значение	Метод проверки
Precision@5	≥ 60%	Тестовый набор из 15 вопросов с эталонными ответами
Latency (время ответа)	≤ 3 сек.	Измерение на 10 последовательных запросах
Размер базы	≥ 80 файлов	Подготовленный датасет с Python-кодом
Язык запросов	Русский + английский	Тестовые запросы на обоих языках





Архитектура системы

Система состоит из трёх компонентов. Все на Python — никакой Java, никакого отдельного фронтенда на React, никакого Docker по умолчанию.

Компонент 1

Индексирующий скрипт

Автономный Python-скрипт `index.py`. Принимает директорию с `.py`-файлами и выполняет три шага: парсинг файлов модулем `ast` на функции и классы, генерация эмбеддингов через `sentence-transformers`, сохранение в векторную БД.

Подсказка: Стратегия чункования — важное архитектурное решение. Простейший вариант: один `chunk` = одна функция или класс. Обоснование выбора стоит явно записать в README — эксперты спросят.

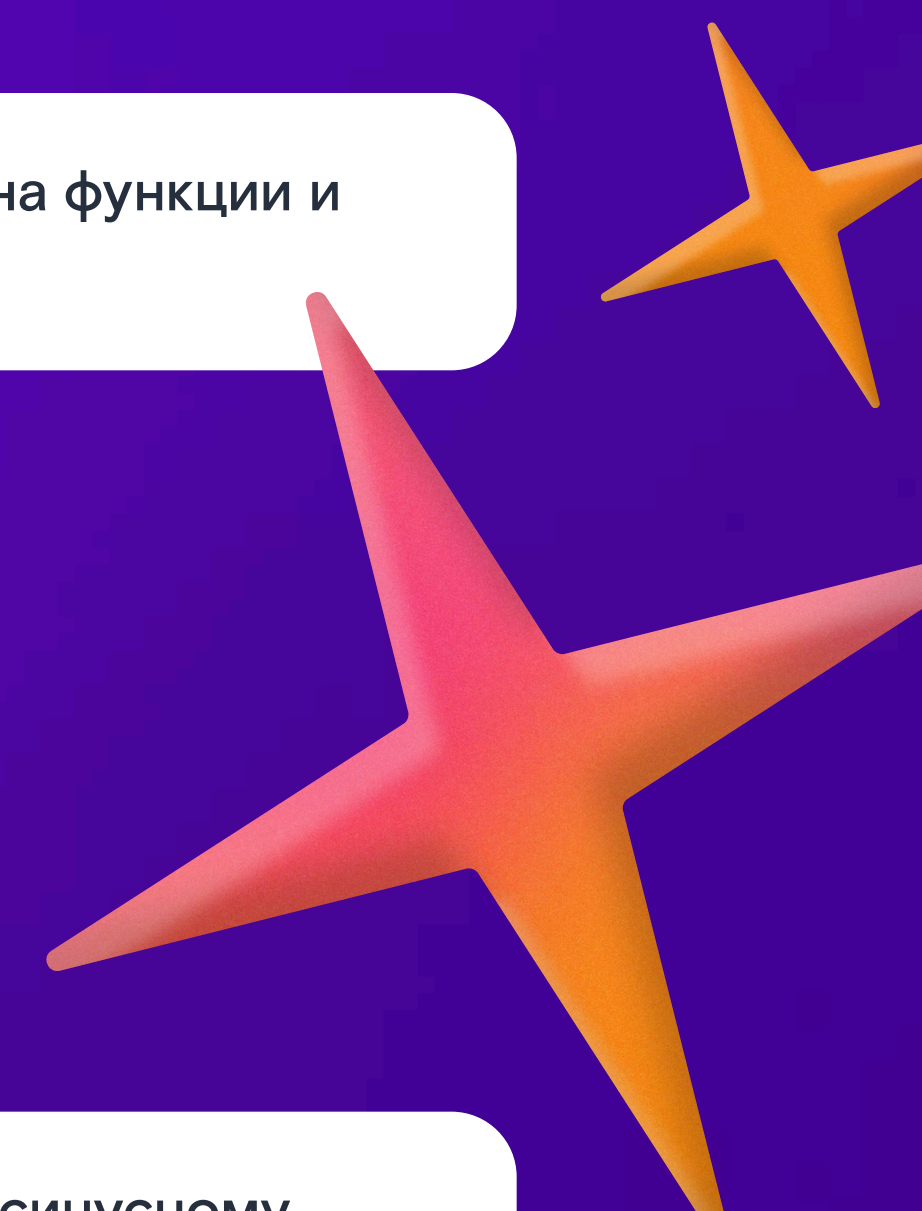
Компонент 2

Векторная база ChromaDB

ChromaDB — это встроенная векторная БД на Python. Хранит эмбеддинги и метаданные, умеет искать ближайших соседей по косинусному расстоянию. Ставится через `pip install chromadb`, работает из коробки без настройки сервера.

Каждый чанк сохраняется с метаданными: путь к файлу, тип (`function/class`), имя, номера строк, `docstring`. При поиске возвращаются топ-K ближайших по вектору, а не по тексту.

Подсказка: ChromaDB можно заменить на FAISS (`pip install faiss-cpu`) если хочется меньше абстракций и больше контроля. Обе библиотеки хорошо подходят, выбор — на команде.





Архитектура системы

Компонент 3

Веб-интерфейс Streamlit

Streamlit — это быстрый способ сделать веб-UI на Python без JavaScript и HTML. Одна страница с полем ввода, кнопкой поиска и списком результатов. Каждый результат — карточка с путём к файлу, кодом с подсветкой синтаксиса и оценкой релевантности в процентах.

Подсказка: В Streamlit подсветка кода — одна строка: `st.code(chunk_content, language="python")`. Ничего дополнительно подключать не нужно.

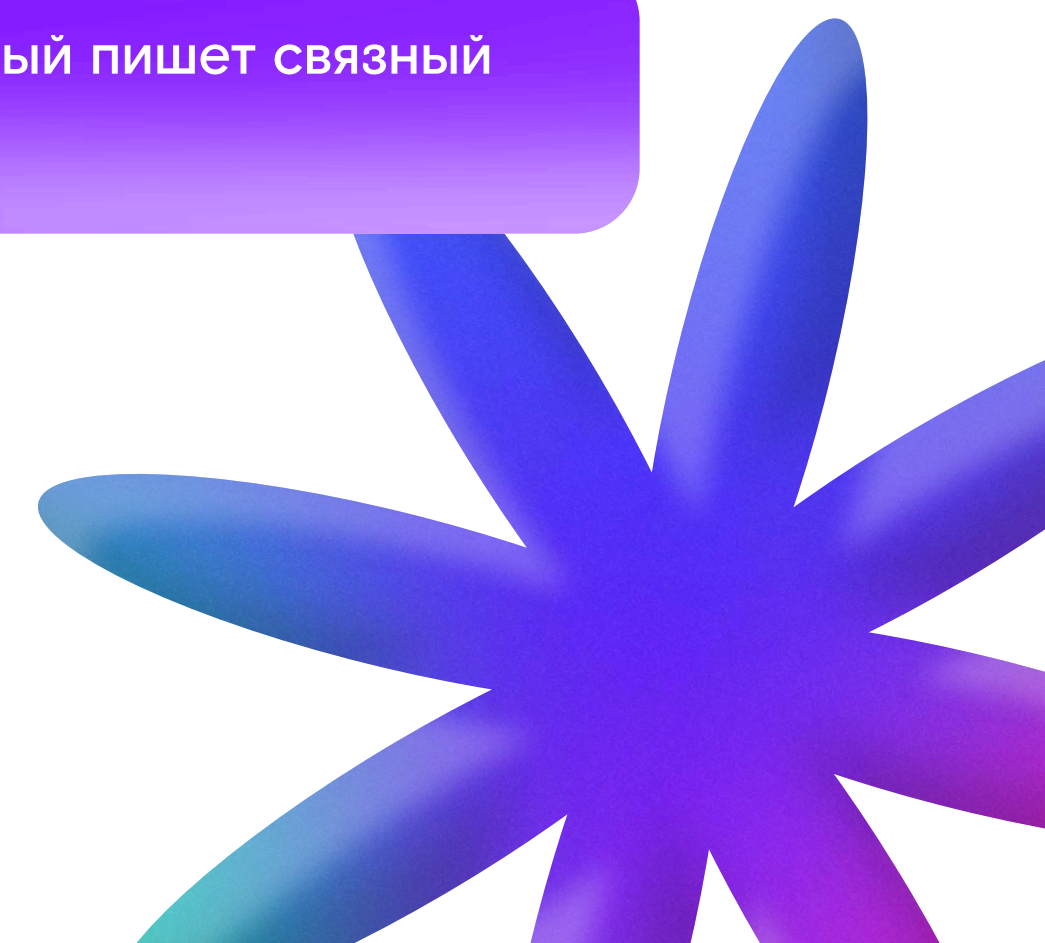
Компонент 4

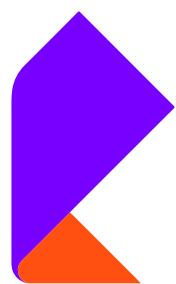
LLM-ответы

Опционально

Если команда хочет сделать полный RAG, добавляется чат-режим: найденные фрагменты + вопрос передаются в LLM, который пишет связный ответ. Модель — на усмотрение команды: Ollama локально (mistral:7b) или любая облачная.

Подсказка: Локальный Ollama поднимается одной командой `ollama serve` и говорит по HTTP. Python-клиент — `pip install ollama`. Если интернета или мощности не хватает — просто пропустите этот компонент, поиск работает и без него.





4. Предоставляемые данные

Датасет	Описание	Объём
codebase_python.zip	Open-source Python-проект: FastAPI сервис с типовой структурой	~80 файлов, ~8К строк
eval_questions.json	Тестовый набор: 15 вопросов с эталонными chunk_id	15 записей
sample_queries.txt	Примеры запросов разной сложности для демо	20 запросов

Участники вправе дополнительно использовать любые открытые Python-репозитории для расширения тестовой базы — это будет учтено при оценке.





Требования к прототипу

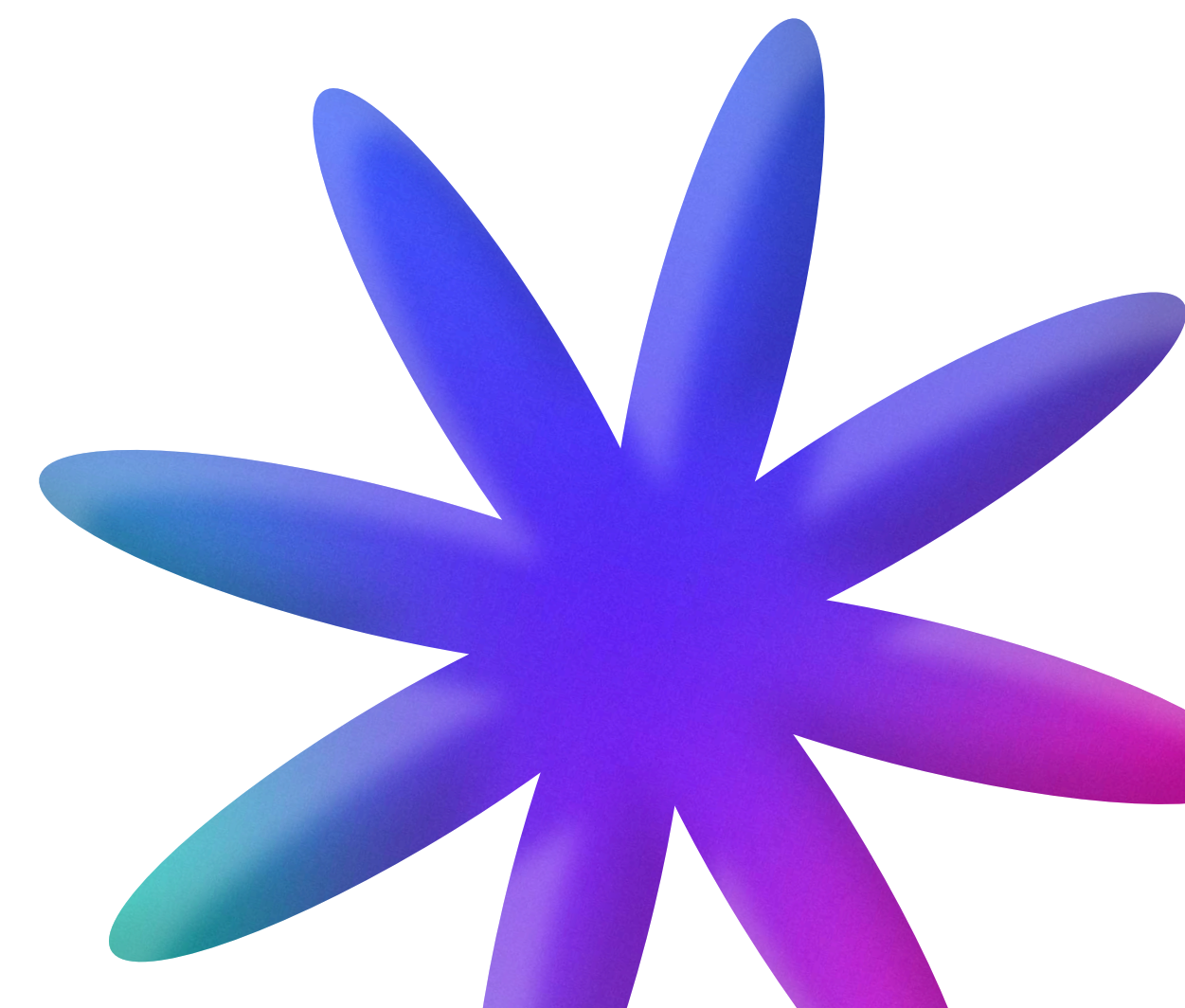
Обязательные

- Индексирующий скрипт корректно режет файлы на функции и классы, сохраняет эмбединги в ChromaDB (или FAISS).
- Семантический поиск возвращает релевантные фрагменты по запросу на русском или английском.
- Веб-интерфейс на Streamlit показывает результаты с подсветкой синтаксиса и оценкой релевантности.
- Система запускается двумя командами: `python index.py <папка>` для индексирования и `streamlit run app.py` для UI.
- README содержит описание архитектуры, инструкцию запуска, 5 примеров запросов с ответами и обоснование стратегии чункования.

Дополнительные

Плюс к оценке

- LLM-ответы: чат-режим с генерацией связного ответа на основе найденных фрагментов.
- Гибридный поиск: комбинация векторного и полнотекстового с настраиваемым весом.
- Метрики качества: Precision@5 вычисляется и показывается на отдельной странице Streamlit.
- Поддержка второго языка (Java или JavaScript) через tree-sitter или регулярки.
- Docker Compose для запуска: `docker compose up` и всё работает.





Технический стек

Слой	Обязательно	Рекомендуется
Язык	Python 3.12	Одна точка зрения на весь проект
Парсинг	Модуль ast (встроенный)	tree-sitter — для второго языка
Эмбеддинги	sentence-transformers	Модель: paraphrase-multilingual-MiniLM-L12-v2
Векторная БД	ChromaDB или FAISS	ChromaDB проще, FAISS быстрее
UI	Streamlit	Никаких React и npm
LLM (опц.)	Ollama или OpenAI API	mistral:7b помещается в 8 ГБ RAM





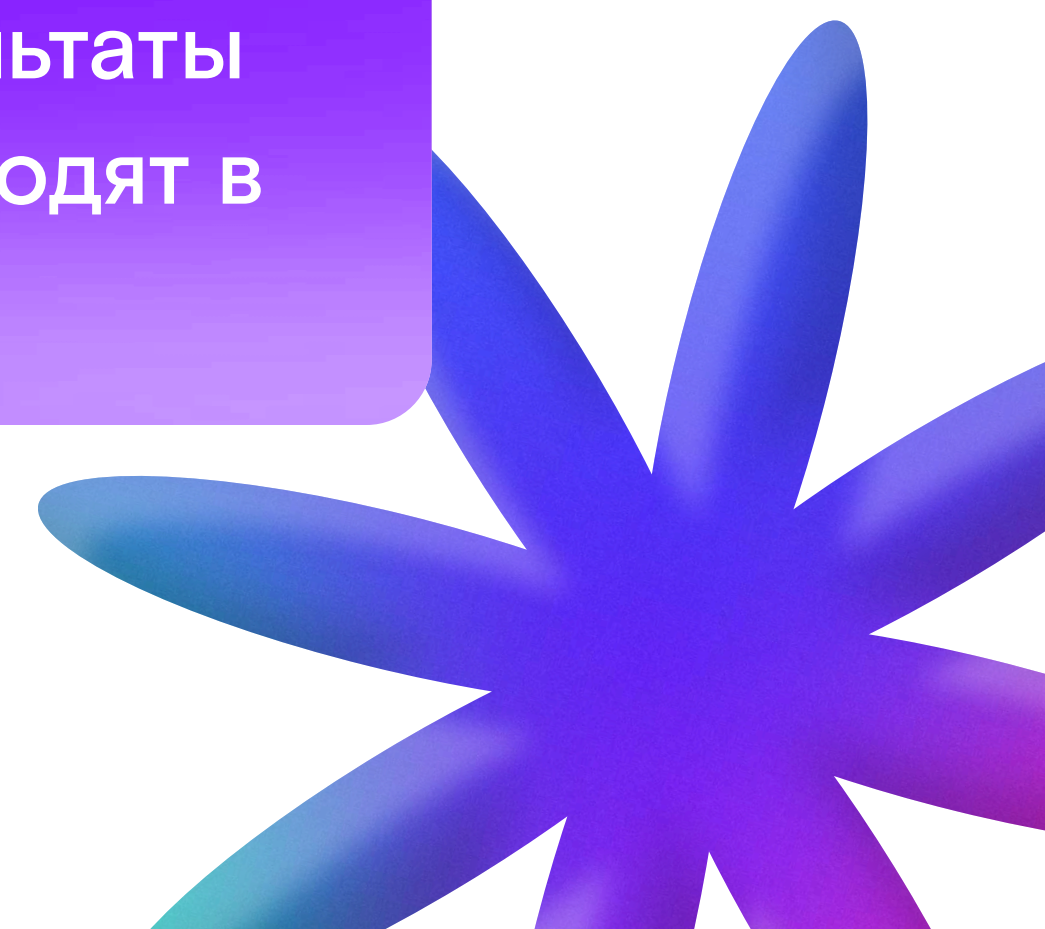
Как это работает

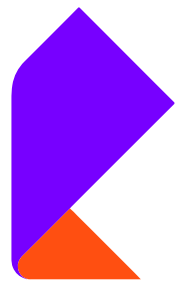
При индексировании

Скрипт обходит директорию, находит .ру-файлы. Каждый файл режется модулем ast на функции и классы. Для каждого фрагмента sentence-transformers генерирует вектор размерностью 384. Вектор и метаданные сохраняются в ChromaDB.

При поиске

Пользователь вводит запрос в Streamlit. Та же модель переводит запрос в вектор. ChromaDB возвращает топ-5 ближайших фрагментов. Streamlit рисует результаты карточками. Если включён LLM-режим — найденные фрагменты и вопрос уходят в Ollama, возвращается связный ответ.





Критерии оценки

Критерий	Вес	Что оценивается
Качество поиска	35%	Precision@5 на eval_questions.json, показанный вживую
Архитектурные решения	25%	Обоснование стратегии чункования, выбора модели и БД
Качество UI	20%	Удобство Streamlit-интерфейса, читаемость кода, время отклика
Документация	20%	Ясность README, схема архитектуры, примеры запросов

Штрафы

- Система не запускается двумя командами — минус 15%.
- Отсутствует подсветка синтаксиса в результатах — минус 5%.





Формат сдачи

Команда публикует код в открытом репозитории GitHub. Защита в ВКС — 10 минут: 7 на демонстрацию, 3 на вопросы. На демо: запуск индексирования, живой поиск по запросам экспертов, показ Precision@5.

Подсказка: Главный вопрос на защите — «почему именно такая стратегия чункования?». Заготовьте ответ в 2–3 предложения. Вариант «режем по функциям, потому что функция — это минимальная семантическая единица Python-кода» работает отлично.

